## EGCI427 Assignment 1: Contact List Web Application

### 1. Overview

Build a full-stack **Contact List** web application with Vue.js 3, Node.js (Express) and MongoDB Atlas, using JavaScript only (no TypeScript). It is worth 20% of your course grade and is due **November 20, 2026 at 10:00**.

A logged-in user can view all contacts as cards, add a new contact, edit and delete an existing one, and search by first or last name. Every read and write goes through your own REST API in `server.js`.

| Item | Detail |
|---|---|
| Course | EGCI427 Web Programming |
| Weight | 20% of the course grade (20 marks) |
| Group size | 1-2 students |
| Released | Wednesday 27 October 2026 |
| Due | Wednesday 18 November 2026, 10:00 (UTC+7) |
| Frontend | Vue 3 + Vite, JavaScript, `http://localhost:8427` |
| Backend | Node.js + Express, `server.js`, `http://localhost:3427` |
| Database | MongoDB Atlas, database `ContactList` |
| Submit | GitHub repository URL on Microsoft Teams, one submission per group |

### Learning outcomes

On completing this assignment, you will be able to:

1. Design and build a REST API with Node.js and Express that performs CRUD operations on MongoDB.
2. Build a single-page application with Vue 3 components, Vue Router and form validation.
3. Connect a frontend to a backend over HTTP and handle success and error responses.
4. Store passwords as hashes and restrict pages to logged-in users.
5. Manage a team project with Git and GitHub.

## 2. Technology stack

Use only the open-source tools below, written in plain JavaScript. TypeScript files (`.ts`, `.vue` with `lang="ts"`) are not accepted and that part of the work scores zero.

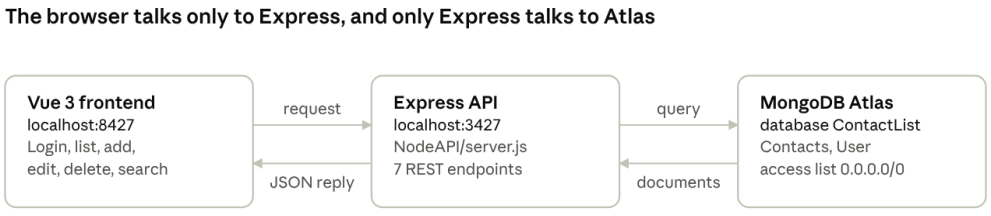
| Layer | Required | Notes |
|---|---|---|
| Runtime | Node.js current LTS (22.x or newer) | Check with `node -v` |
| Frontend framework | Vue 3 with Vite | Create with `npm create vue@latest`; answer **No** to TypeScript |
| Routing | Vue Router 4 | Select it when creating the project |
| UI library (choose one) | Fomantic UI for Vue, Vuetify 3 or Bootstrap 5 | Plain CSS is allowed for small adjustments |
| HTTP client | `axios` or the built-in `fetch` | Pick one and use it consistently |
| Backend framework | Express | Entry file must be `server.js` |
| Database driver | `mongoose` or the official `mongodb` driver | |
| Other backend packages | `cors`, `dotenv` | MD5 via Node's built-in `crypto` module or the `md5` package |
| Database | MongoDB Atlas (free M0 cluster) | |
| Version control | Git + a private GitHub repository | |

Sample avatar images are available on the Semantic UI card page; for example `https://semantic-ui.com/images/avatar2/large/kristy.png`. You may also use any image URL you have the right to use.

---

## How the parts connect

The browser only ever talks to your Express server; it never connects to MongoDB direct




**Application architecture** · 3 tiers, 2 ports

Each user action in Vue becomes one HTTP request to Express, which runs one MongoDB query and returns JSON.

---

## 3. Naming rules and project structure

Pick **one** group member's student ID and use its last 4 digits everywhere below. In this handout the example ID ends in `2345`. Wrong names cost **5 marks**, so check them before you submit.

| What | Required name | Example |
|---|---|---|
| GitHub repository | `Contact<last 4 digits>` | `Contact2345` |
| Vue project folder | `Contact<last 4 digits>` | `Contact2345` |
| Backend folder | `contactapi` | `contactapi` |
| Backend entry file | `server.js` | `contactapi/server.js` |
| Database | `ContactList` | |
| Contact collection | `Contacts` | |
| User collection | `User` | |
| Frontend port | 8427 | `http://localhost:8427` |
| Backend port | 3427 | `http://localhost:3427` |

npm requires lowercase package names, so `package.json` may show `"name": "contact2345"`. That is fine; the **folder** and **repository** must still be `Contact2345`.

Mongoose pluralises and lowercases model names by default (a model `User` becomes collection `users`). Pass the collection name explicitly, for example `mongoose.model('User', userSchema, 'User')` and `mongoose.model('Contact', contactSchema, 'Contacts')`.

---

### Required repository layout

```
Contact2345/                  <- repository root
├── README.md
├── Contact2345/              <- Vue project (frontend, port 8427)
│   ├── .gitignore
│   ├── package.json
│   ├── vite.config.js
│   ├── index.html
│   └── src/
│       ├── main.js
│       ├── App.vue
│       ├── router/index.js
│       ├── services/api.js   <- all HTTP calls in one place
│       └── views/
│           ├── LoginView.vue
│           ├── ContactListView.vue
│           ├── NewContactView.vue
│           └── EditContactView.vue
└── contactapi/                  <- backend (port 3427)
    ├── .gitignore
    ├── .env.example
    ├── package.json
    ├── server.js
    ├── models/               <- optional: Contact.js, User.js
    ├── routes/
    └── controllers/
```

File names inside `src/` and `models/` are a recommendation, not a requirement. The top-level folder names, `server.js` and the ports are required.

---

## 4. Step-by-step instructions

Work in this order: database first, then the API, then the Vue pages. Test every API endpoint on its own before you connect the frontend to it.

---

### Step 1. Set up MongoDB Atlas (1 mark)

1. Create a free MongoDB Atlas account and an **M0** cluster.
2. Under **Database Access**, create a database user for this app (for example `contactapp`) with read and write access.
3. Under **Network Access**, click **Add IP Address** and set the Access List Entry to `0.0.0.0/0` with the comment `wildcard IP address`. Leave "This entry is temporary" **off**, or the grader cannot connect.
4. Create the database `ContactList` with two collections: `Contacts` and `User`.
5. Copy the Node.js connection string, for example `mongodb+srv://contactapp:<password>@cluster0.xxxxx.mongodb.net/ContactList`.
6. Create the login account. Generate the MD5 hash of the password in a terminal:

   ```
   node -e "console.log(require('crypto').createHash('md5').update('12344321').digest('hex'))"
   ```

   Insert this document into `User`, pasting the 32-character hash you got:

   ```
   { "username": "bob", "password": "<32-character MD5 hash of 12344321>" }
   ```
7. Insert at least **4 sample contacts** into `Contacts` so the list page has data to show.

Each document must use these field names exactly, because the API and the grader rely on them:

| Collection | Field | Type | Rule |
|---|---|---|---|
| Contacts | `contactId` | String | Required, unique, cannot be changed after creation |
| Contacts | `firstName` | String | Required |
| Contacts | `lastName` | String | Required |
| Contacts | `mobile` | String | Required, digits only, 9 or 10 digits |
| Contacts | `email` | String | Optional, valid email format when given |
| Contacts | `imageURL` | String | Optional; show a default avatar when empty |
| User | `username` | String | Required, unique |
| User | `password` | String | Required, stored as an MD5 hex hash, never plain text |

MD5 is used here only to practice hashing. It is not safe for real passwords; production systems use slow, salted algorithms such as bcrypt or Argon2.

---

### Step 2. Build the backend in `contactapi/`

1. Create the folder and install packages:

   ```
   mkdir contactapi && cd contactapi
   npm init -y
   npm install express mongoose cors dotenv
   ```

2. In `package.json`, add the script `"start": "node server.js"`. The grader runs the API with `npm start`.
3. Create `.env` (never committed) and `.env.example` (committed, placeholders only):

   ```
   MONGODB_URI=mongodb+srv://<user>:<password>@<cluster>/ContactList
   PORT=3427
   ```

4. Add `.env` to `Contactapi/.gitignore`. The sample `.gitignore` in Section 7 ignores `.env.local` but **not** `.env`, so add that line yourself.
5. In `server.js`: load `dotenv`, enable `express.json()` and `cors` for `http://localhost:8427`, connect to Atlas, define the two models, add every route in Section 5, then listen on port 3427 and log `API running on http://localhost:3427`.
6. Validate input on the server too, not only in the browser. Return the status codes in Section 5 and a JSON body such as `{ "message": "Contact ID already exists" }` on errors.
7. Test every endpoint with Postman, Thunder Client or `curl` before starting the frontend.

---

### Step 3. Create the Vue project

1. From the repository root run `npm create vue@latest`. Enter the project name `Contact2345`, answer **No** to TypeScript and **Yes** to Vue Router. Other options are up to you.
2. Install your chosen UI library and follow its Vue 3 setup guide.
3. In `vite.config.js` set the port: `server: { port: 8427, strictPort: true }`.
4. Put all HTTP calls in `src/services/api.js` with the base URL `http://localhost:3427`, so no component hard-codes a URL.
5. Define four routes: `/login`, `/` (contact list), `/contacts/new` and `/contacts/:id/edit`.
6. Add a navigation guard that sends anyone who is not logged in to `/login`.
7. Show the same top bar on every page: logo, a **Contacts** link and a user icon. When logged in, the user icon offers **Logout**.

---

### Step 4. Implement the pages

Match the look of Figures 1 to 4 below closely. It does not need to be pixel-identical.

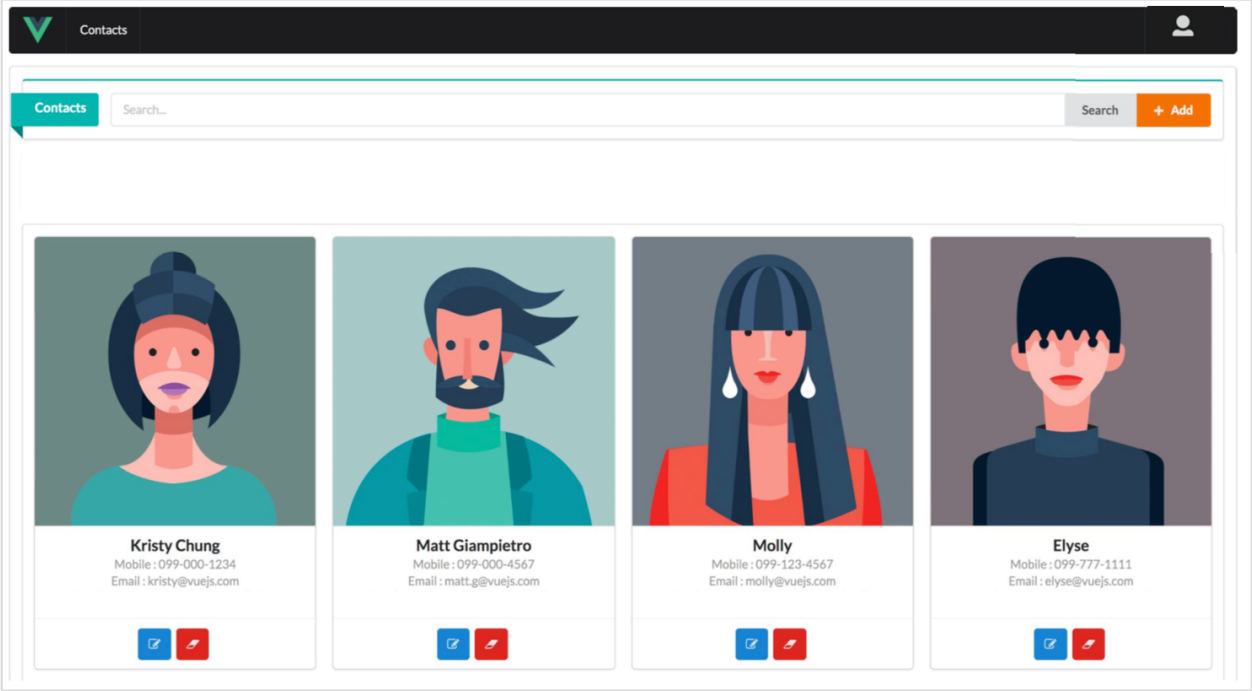



**Figure 1** · contact list page

Each card shows the image, full name, mobile and email, with a blue **Edit** and a red **Delete** button. Use your own avatar images in place of the grey placeholders.

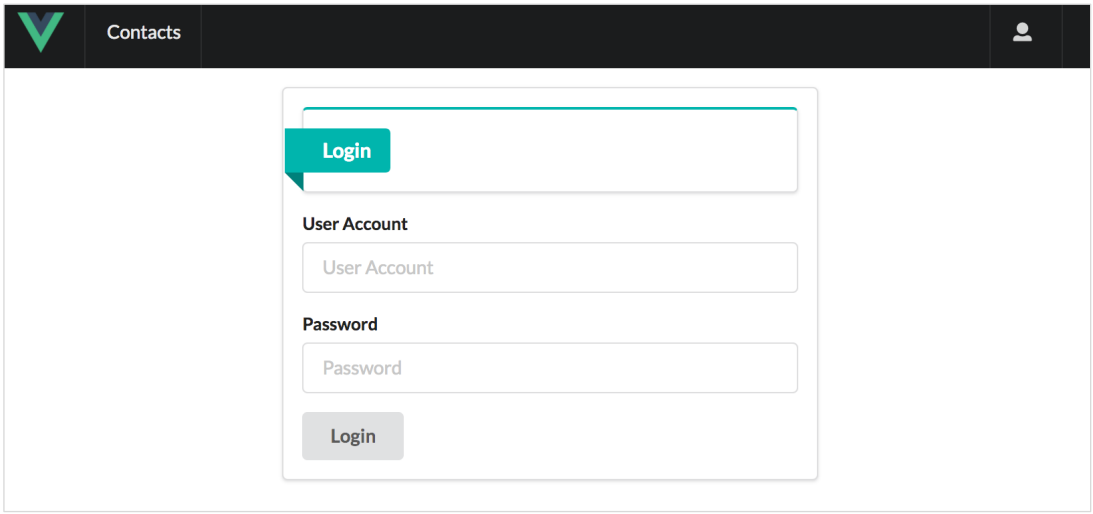
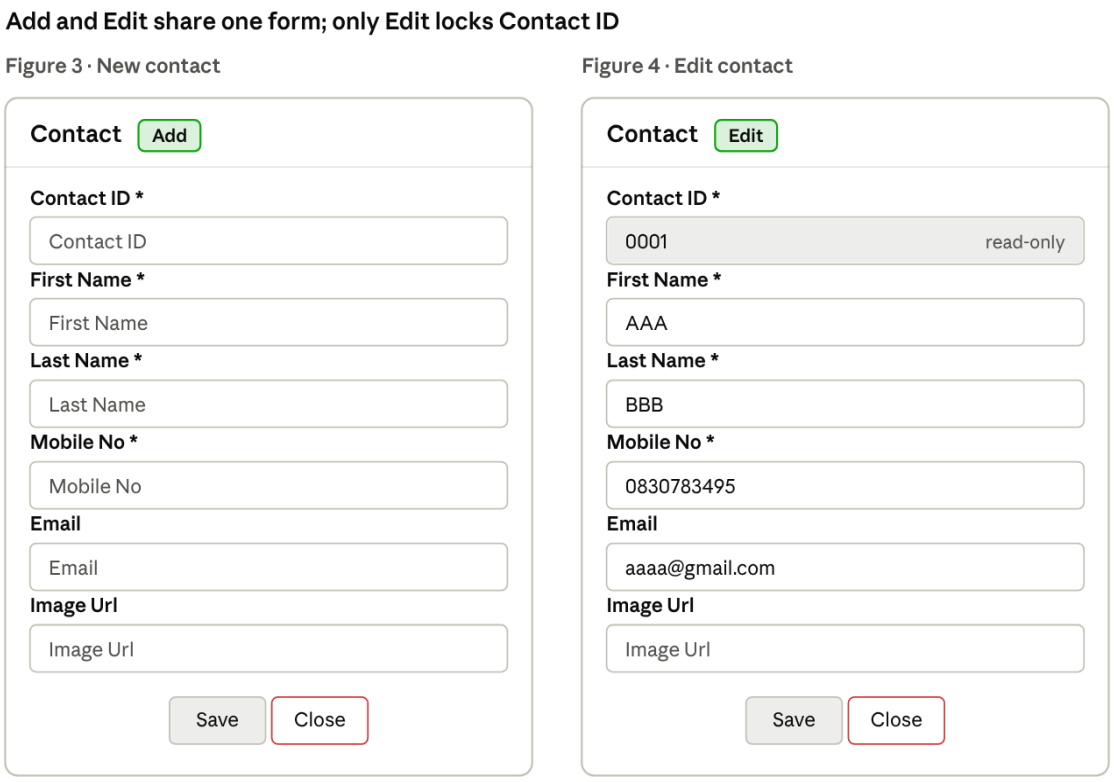



**Figure 2** · login page




**Figures 3 and 4** · new and edit contact forms

Fields marked * are required. On the Edit form, Contact ID is filled in and cannot be changed.

### 4.1 Login page (2 marks)

- Fields: **User Account**, **Password** (masked) and a **Login** button.
- On submit, call `POST /users/login`. The server hashes the submitted password with MD5 and compares it with the stored hash.
- Success: save the logged-in state (for example the username in `localStorage`) and go to the contact list.
- Failure: show "Invalid username or password" and stay on the page. Do not say which of the two was wrong.
- Typing any other URL while logged out must redirect to `/login`. **Logout** clears the state and returns to `/login`.

### 4.2 Contact list page (2 marks)

- Header row: a **Contacts** label, a search box, a grey **Search** button and an orange **+ Add** button.
- Load contacts from `GET /contacts/getcontacts` and show one card each in a responsive grid: image, full name, mobile, email, a blue **Edit** button and a red **Delete** button.
- Show a default avatar when `imageURL` is empty or the image fails to load, and a message when there are no contacts.

### 4.3 New contact page (2 marks)

- **+ Add** opens a form with Contact ID*, First Name*, Last Name*, Mobile No*, Email and Image Url, plus **Save** and **Close** buttons (* = required).
- Validate in the browser: required fields filled, mobile is 9 or 10 digits, email has a valid format when entered. Show the error next to the field.
- **Save** calls `POST /contacts/add`. On success return to the list, where the new card appears. A duplicate Contact ID shows the server's error message.
- **Close** returns to the list without saving.

### 4.4 Edit contact page (3 marks)

- The blue **Edit** button opens the same form filled with that contact's data from `GET /contacts/getcontact/:id`.
- **Contact ID is shown but cannot be edited** (read-only or disabled). The server must also refuse to change it.
- First Name, Last Name and Mobile No are required, with the same validation as 4.3.
- **Save** calls `PUT /contacts/update/:id`. On success return to the list, which shows the new values.

### 4.5 Delete (2 marks)

- The red **Delete** button asks for confirmation, for example "Delete Molly from your contacts?".
- On confirm, call `DELETE /contacts/delete/:id`. The card disappears without a full page reload, and the document is gone from Atlas.
- On cancel, nothing changes.

### 4.6 Search (2 marks)

- Clicking **Search** or pressing Enter calls `GET /contacts/search?q=<text>`.
- The server returns contacts whose **first name or last name** contains the text, ignoring case. For example, `mol` finds Molly and `CHUNG` finds Kristy Chung.
- An empty search shows all contacts. No match shows "No contacts found".
- Search must run on the server with a MongoDB query (for example a case-insensitive `$regex` with `$or`). Escape regex special characters in the user's text. Filtering the array only in the browser earns at most 1 of the 2 marks.

---

## 5. REST API specification (5 marks)

`Contactapi/server.js` must provide all seven endpoints below at base URL `http://127.0.0.1:3427`, with these exact methods and paths. In every path, `:id` is the contact's **`contactId`** (for example `0001`), not MongoDB's `_id`.

| # | Method | Path | Request body | Success | Errors |
|---|---|---|---|---|---|
| 1 | POST | `/users/login` | `username`, `password` | 200 + `{ username }` | 400 missing field, 401 wrong username or password |
| 2 | GET | `/contacts/getcontacts` | none | 200 + array of all contacts | 500 server error |
| 3 | GET | `/contacts/getcontact/:id` | none | 200 + one contact | 404 not found |
| 4 | GET | `/contacts/search?q=<text>` | none | 200 + array (may be empty) | 500 server error |
| 5 | POST | `/contacts/add` | contact JSON (below) | 201 + the saved contact | 400 invalid data, 409 duplicate `contactId` |
| 6 | PUT | `/contacts/update/:id` | contact JSON | 200 + the updated contact | 400 invalid data or changed `contactId`, 404 not found |
| 7 | DELETE | `/contacts/delete/:id` | none | 200 + `{ message }` | 404 not found |

Contact JSON for endpoints 5 and 6:

```
{
  "contactId": "0001",
  "firstName": "Molly",
  "lastName": "Smith",
  "mobile": "0991234567",
  "email": "molly@vuejs.com",
  "imageURL": "https://semantic-ui.com/images/avatar2/large/molly.png"
}
```

---

## 6. Marking criteria and rubric

The assignment is marked out of **20**, which equals 20% of the course grade. Marks are awarded only for features the grader can run and see working.

### Summary

| # | Criterion | Marks |
|---|---|---|
| 1 | MongoDB Atlas setup | 1 |
| 2 | Login page | 2 |
| 3 | Contact list page and UI | 2 |
| 4 | New contact page | 2 |
| 5 | Edit contact page | 3 |
| 6 | Delete | 2 |
| 7 | Search | 2 |
| 8 | REST API (`server.js`) | 5 |
| 9 | Code quality, README and Git use | 1 |
| | **Total** | **20** |

---

### How your work will be graded

1. The grader clones your repository and creates `Contactapi/.env` from the credentials in your README.
2. In `Contactapi`: `npm install`, then `npm start`.
3. In `Contact2345`: `npm install`, then `npm run dev`, and opens `http://localhost:8427`.
4. The grader logs in as `bob` / `12344321` and tests each criterion in the order of the rubric.

If the project does not start with these commands, the grader spends at most 10 minutes trying to fix setup problems. Anything that still cannot run scores 0, so test a fresh clone yourself before the deadline.

---

## 7. How to submit

Submit by November 18, 2026 at **10:00 (UTC+7)**. Missing **any** of steps 1 to 6 gives 0 for the whole assignment.

### Steps

1. Create a **private** GitHub repository named `Contact<last 4 digits>`, for example `Contact2345`.
2. Add the instructor as a collaborator: GitHub user **lalitanar** (lalita.nar@mahidol.edu). Every group member should also be a collaborator.

3. Copy the `.gitignore` below into **both** `Contact2345/` and `Contactapi/`, and add the line `.env` to the `Contactapi` copy. Run `git status` and confirm that `node_modules/` and `.env` are not listed.
4. Push the frontend (`Contact2345/`) and the backend (`Contactapi/`) to the `main` branch.
5. Complete `README.md` at the repository root using the template below.
6. One member posts **only the GitHub repository URL** in the Microsoft Teams assignment. One submission per group.

## README.md template

```
# Contact2345 – EGCI427 Assignment 1

Repository: https://github.com/<owner>/Contact2345

## Group members
| Student ID | Full name |
| --- | --- |
| 6xxxxxx2345 | Firstname Lastname |
| 6xxxxxxxxxx | Firstname Lastname |

## How to run
1. cd Contactapi && npm install && npm start      (API on http://localhost:3427)
2. cd Contact2345 && npm install && npm run dev  (app on http://localhost:8427)
3. Log in with bob / 12344321

## Database (for grading only)
MONGODB_URI=mongodb+srv://<user>:<password>@<cluster>/ContactList

## UI library used
Fomantic UI / Vuetify / Bootstrap

## Who did what
- Member 1: ...
- Member 2: ...

## Additional notes
Anything the grader should know (known issues, extra features).
```

The repository is private, so the database credentials may go in the README for grading. Use a database user created only for this assignment, never a password you use anywhere else, and delete that user after grades are released.

## `.gitignore` file

```
.DS_Store
node_modules/
/dist/

# local env files
.env
.env.local
.env.*.local

# Log files
```

```
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Editor directories and files
.idea
.vscode
*.suo
*.ntvs*
*.njsproj
*.sln
*.sw*
```

## Final checklist

- ☐ Repository and Vue folder are named `Contact<last 4 digits>`
- ☐ Repository is private and **lalitanar** has accepted the collaborator invitation
- ☐ `Contactapi/server.js` starts with `npm start` on port 3427
- ☐ Frontend starts with `npm run dev` on port 8427
- ☐ No TypeScript anywhere in the project
- ☐ Atlas access list contains `0.0.0.0/0` and is not temporary
- ☐ `bob` / `12344321` logs in, with the password stored as MD5
- ☐ All 7 API endpoints answer as in Section 5
- ☐ `node_modules/` and `.env` are not in the repository
- ☐ README has the repository URL, every member's student ID and full name, and run instructions
- ☐ A fresh `git clone` into an empty folder runs by following only the README
- ☐ Repository URL posted on Microsoft Teams before 10:00 on 18 November 2026

## Deadline and late work

- The submission time is the later of the Teams post and the last commit pushed to `main`. Commits pushed after the deadline count as late.
- Late work loses 10% of the earned marks per day or part of a day, for up to 3 days. Work more than 3 days late scores 0.
- If illness or an emergency stops you from submitting on time, contact the instructor **before** the deadline, with evidence.

## Academic integrity

- Each group writes its own code. Sharing code between groups, or submitting code copied from another group or an online project, is plagiarism and will be reported under university regulations.
- You may use official documentation, tutorials and AI assistants to learn, but every member must understand and be able to explain every part of the submission.
- Each member should commit their own work from their own GitHub account, so the history shows who built what. **Marks will be estimated according to the proportion of the committed codes.**